Perfecto. Voy a darte un plan **ultra detallado de 6 meses (24 semanas)** para maximizar tus probabilidades de conseguir trabajo y llegar con nivel “industria”, basado en lo que tu documento prioriza: **SQL serio + estadística aplicada + MLOps/producción + GenAI (RAG/LLMs) + portafolio end-to-end** .

---

0) Reglas del juego (para que funcione en 6 meses)

Horas mínimas recomendadas

Para que sea realista en 6 meses, apunta a **12–20 h/semana (modo intensivo)** .
Si estás por debajo de 10 h/sem, se puede, pero el “job-ready” suele tardar más.

Tu stack objetivo (lo que vas a dominar de verdad)

Tu documento define un stack “empleable” 2026 con foco en:

- **Python** (pandas/numpy/sklearn + gráficas)
- **SQL de verdad**: joins + window functions + CTEs + optimización básica
- **Estadística aplicada**: inferencia, A/B, métricas, leakage, sesgos
- **Tabular ML fuerte**: XGBoost/LightGBM + validación seria + SHAP
- **Producción/MLOps**: FastAPI + Docker + MLflow + tests + CI/CD
- **GenAI**: RAG con LlamaIndex/LangChain + evaluación/observabilidad
- Extra muy rentable: **dbt + warehouse (BigQuery/Snowflake/Redshift)**

Los 3 proyectos “mínimos” del portafolio (no negociable)

Tu documento lo deja clarísimo: tu portafolio debe probar **punta a punta** :

1. **End-to-end ML en producción**
   Dataset → training pipeline → MLflow → FastAPI → Docker → CI → monitoreo básico
2. **Proyecto de negocio (KPI + experimento)**
   Define KPI, baseline, segmentación, análisis de error y recomendación accionable
3. **RAG/LLM app útil**
   “Chat con documentos” + evaluación (precision/recall retrieval) + control de fuentes

Y además: **escribir 1–2 páginas por proyecto** (tipo case study) .

---

1) Rutina semanal (plantilla) – 15 h/semana

Distribución (L–V + S)

- **SQL diario (45–60 min)**: window functions sí o sí
- **Bloque principal (60–90 min)**: ML / MLOps / GenAI según la fase
- **Sábado (3–5 h)**: construir proyecto + documentación
- **Domingo (30–45 min)**: retro + plan de la semana (checklist)

Output semanal obligatorio (para que no “estudies en el aire”)

Cada semana debes producir 1 cosa visible:

- 1 notebook limpio o script modular
- o 5–10 queries SQL guardadas y explicadas
- o un endpoint nuevo en FastAPI
- o un experimento trackeado en MLflow
- o una mejora clara al README/case study

Esto aplica la regla del documento: **portafolio > certificados** .

---

2) Plan de 6 meses (24 semanas) – ultra estructurado

> El orden sigue la “ruta sugerida” del documento: fundamentos → ML industria → MLOps/producción → especialización (GenAI/RAG) .

MES 1 (Semanas 1–4): Fundamentos “profesionales” + SQL serio

Semana 1 — Setup profesional + baseline

Objetivo: que tu entorno y forma de trabajar ya parezca “trabajo real”
Checklist:

- Repo “career-2026” en GitHub con:
  - `/notes/sql/` , `/notes/stats/` , `/projects/`
- Entorno: conda/uv/poetry + estructura limpia de notebooks
- Plantilla de README (qué problema, datos, métrica, resultados, cómo correr)

SQL (diario):

- JOINs (INNER/LEFT), GROUP BY, HAVING, subqueries
- 2 ejercicios/día → guardar query + 3 bullets de “por qué”

Entrega de la semana:

- “SQL Cheatsheet v1” + 15 queries comentadas.

Semana 2 — SQL avanzado (window functions) + mini-proyecto SQL

Objetivo: dominar window functions + CTEs
SQL foco:

- ROW_NUMBER / RANK / DENSE_RANK / LAG / LEAD
- CTEs y “query readability”
- Optimización básica: evitar SELECT *, filtros temprano

Mini-proyecto (2–3 días):

- Dataset público tipo e-commerce/logística
- Preguntas de negocio:
  - top productos por región/mes
  - retención simple (cohortes)
  - ventas acumuladas (running totals)

Entrega:

- 1 notebook o .md con:
  - 20–25 queries “de portafolio”
  - y explicaciones tipo “para qué sirve en negocio”.

Semana 3 — Estadística aplicada (para decisiones reales)

Objetivo: inferencia + A/B + métricas (sin humo)
Temas:

- intervalos de confianza, p-values, power básico

- sesgo-varianza (qué implica en overfitting)
- leakage: cómo se ve y cómo se evita

Práctica:

- simular un A/B test con Python:
  - métrica primaria
  - supuestos
  - decisión final

Entrega:

- Notebook: “AB testing from scratch” + conclusiones “para negocio”.

Semana 4 — ML clásico “bien hecho” (pipeline real)

Objetivo: dejar de “entrenar modelos” y empezar a construir solución
Temas:

- scikit-learn Pipelines + ColumnTransformer
- validación: CV / time split según el caso
- métricas ligadas a negocio (no solo accuracy)

Entrega:

- Starter del Proyecto 1 (estructura lista + baseline entrenado).

MES 2 (Semanas 5–8): ML “de industria” + Proyecto 1 (tabular potente)

Semana 5 — Feature engineering + baseline serio

Objetivo: que tu baseline sea difícil de tumbar

- features (fechas, ratios, agregaciones)
- manejo de desbalance (si aplica)
- logging básico de experimentos

Entrega:

- baseline v1 + comparación contra “dummy” + análisis de error.

Semana 6 — XGBoost/LightGBM + tuning responsable

Tu doc lo marca como imprescindible en tabular .

- early stopping, valid split correcto
- tuning pequeño pero justificado (no grid search infinito)
- calibración si aplica

Entrega:

- modelo v2 + tabla de experimentos (qué cambiaste y por qué).

Semana 7 — Interpretabilidad + “por qué falla”

- SHAP + error analysis
- métricas por segmento (región, canal, cohortes)

Entrega:

- reporte de interpretabilidad + “riesgos / sesgos / leakage checks”.

Semana 8 — Convertirlo en “producto” (Proyecto 1 listo para MLOps)

Objetivo: que el proyecto ya huela a producción

- limpiar repo (src/, configs/, notebooks/ solo exploración)
- README fuerte + 1–2 páginas (case study)

Entrega:

- Proyecto 1 v1 (ML “industria”) + case study v1.

MES 3 (Semanas 9–12): MLOps / Producción de verdad (Proyecto 1 → producción)

Tu documento define una salida mínima clarísima:
✅ **API con FastAPI + Docker + MLflow + tests + CI simple**

Semana 9 — MLflow tracking + “reproducible training”

- tracking de runs, params, métricas
- guardar modelo + artefactos
- script `train.py` reproducible (misma data → mismo resultado)

Entrega:

- MLflow corriendo local + experimentos registrados.

Semana 10 — FastAPI (serving) + contrato de API

- endpoint `/predict`
- validación de inputs (pydantic)
- versionado simple de modelo

Entrega:

- API funcional con ejemplo curl + documentación.

Semana 11 — Docker + pruebas

- Dockerfile + docker-compose (si usas MLflow server local)
- pytest para:
  - features
  - endpoint
  - “data sanity checks”

Entrega:

- `make test` o comando único que corra pruebas.

Semana 12 — CI/CD + monitoreo básico (lo que te separa de junior)

Tu doc lo pide explícito: GitHub Actions + monitoreo drift/data quality

- pipeline CI: lint + tests + build docker
- monitoreo: métricas de input + drift (aunque sea simple)

Entrega final MES 3:

- Proyecto 1 “end-to-end en producción” completo (con CI y monitoreo básico) .

MES 4 (Semanas 13–16): Proyecto 2 (negocio + KPI + experimento) + capa Analytics

Este proyecto prueba que sabes **traducir negocio → hipótesis → decisión** .

Semana 13 — Definir KPI + baseline + “qué decisión se tomará”

- escoger un caso: churn / demanda / fraude / pricing (tu doc sugiere estos)
- escribir:
  - objetivo
  - KPI primario y secundarios
  - “qué haría el negocio si sube/baja”

Entrega:

- 1-pager del problema (muy claro).

Semana 14 — Segmentación + análisis de error + recomendación

- segmentación (cohortes / clusters simple si aplica)
- “dónde funciona / dónde no”
- recomendación accionable

Entrega:

- memo de recomendación (como si fuera para un gerente).

Semana 15 — Dashboard (Power BI/Looker/Tableau) para “contarlo”

Muchísimas vacantes piden dashboarding + KPIs (Power BI/Looker/Tableau) .

- crea un dashboard con:
  - KPI principal
  - breakdown por segmento
  - “alertas” (si cae X%)

Entrega:

- dashboard + README de cómo leerlo.

Semana 16 — Case study final (1–2 páginas)

Tu doc lo exige por proyecto .
Estructura obligatoria:

- Problema
- Datos (y limitaciones)
- Método
- Métrica y resultados
- Decisión recomendada
- Riesgos / siguientes pasos

MES 5 (Semanas 17–20): Proyecto 3 GenAI (RAG) – el diferenciador 2026

Tu documento dice que en 2026 es muy valioso sumar **GenAI/RAG/LLMOps** , y propone RAG con LlamaIndex/LangChain .

Semana 17 — RAG base (ingesta → embeddings → retrieval → respuesta)

- elegir dataset de documentos (PDFs, docs técnicos, normativa, etc.)
- pipeline:
  - chunking
  - embeddings
  - vector DB
  - retrieval + respuesta

Entrega:

- demo CLI o notebook que responda preguntas con citas.

Semana 18 — Evaluación (lo que casi nadie hace)

Tu doc lo pide tal cual: evaluar retrieval con precision/recall y control de fuentes .

- arma un set de 30–50 preguntas con “gold passages”
- mide:
  - recall@k del retrieval
  - exactitud de citas (¿la respuesta usa las fuentes correctas?)
- registra resultados (tabla en README)

Entrega:

- “eval harness” + reporte corto.

Semana 19 — App usable (Streamlit/Gradio) + observabilidad

Tu doc sugiere dashboard/app interactiva como plus .

- UI: chat + panel de “fuentes”
- logging:
  - latencia
  - costo estimado (si aplica)
  - tasa de “no sé”

Entrega:

- app corriendo local con instrucciones 1 comando.

Semana 20 — Hardening (guardrails + calidad)

- respuestas seguras (cuando no hay evidencia, decirlo)
- límites: tamaño docs, timeouts
- tests básicos (smoke tests)

Entrega final MES 5:

- Proyecto 3 (RAG) listo + case study.

MES 6 (Semanas 21–24): Preparación laboral agresiva + entrevistas + aplicaciones

Semana 21 — Portafolio “presentable” (home page)

Tu marca: GitHub + posts + claridad .
Checklist:

- README principal con 3 proyectos (links + 2 líneas de impacto)
- 3 case studies (PDF o markdown)
- 1 video corto (2–4 min) mostrando cada proyecto (opcional pero potente)

Semana 22 — CV orientado a impacto + LinkedIn

En muchas vacantes importan KPIs, dashboards, stakeholders .
Tu CV debe tener bullets tipo:

- “Construí X, redujo Y, medido por Z”
- “Deploy con FastAPI+Docker+MLflow+CI”
- “RAG con evaluación retrieval precision/recall”

Semana 23 — Entrevistas técnicas (rutina diaria)

Todos los días:

- 45–60 min SQL (window functions + casos)

- 30 min ML: leakage, validación, métricas negocio
- 30 min sistema: “cómo lo desplegaste y monitoreaste”

Simula 3 historias:

1. Proyecto 1: “lo desplegué, lo monitoreé, reproducible”
2. Proyecto 2: “KPI → hipótesis → recomendación”
3. Proyecto 3: “RAG con evaluación y control de fuentes”

Semana 24 — Aplicación masiva inteligente (y ajuste)

Estrategia:

- 5–10 aplicaciones/día (personalizadas con 2–3 líneas)
- prioriza vacantes que pidan lo que ya demostraste:
  - SQL + dashboards (muy común)
  - pipelines/ETL/analytics engineering si hiciste dbt
  - producción/ML Ops si tu Proyecto 1 está sólido

Entrega final MES 6:

- Portafolio pulido + CV final + rutina de entrevistas + pipeline de aplicaciones.

---

3) Cómo saber si vas “en ruta” (KPIs personales)

A final de:

- **Mes 1:** haces SQL con window functions sin mirar ayuda (y lo explicas).
- **Mes 3:** tienes el **end-to-end** con FastAPI+Docker+MLflow+tests+CI
- **Mes 5:** tienes RAG con **evaluación** (no solo demo bonita)
- **Mes 6:** tus 3 case studies existen y se leen en 3–5 minutos cada uno

---

4) Si quieres que te lo deje aún más “militar”

Puedo convertir este plan en:

- **calendario día-a-día (180 días)**,

- checklist imprimible por semana,
- y una tabla tipo “trello” con tareas y entregables.

Perfecto. Aquí tienes el plan **“militar” día-a-día (24 semanas / 6 meses)**, pensado para que en 6 meses puedas competir por vacantes con un perfil sólido: **SQL serio + estadística aplicada + ML industria + MLOps/producción + GenAI/RAG + portafolio end-to-end** .

Asumo un ritmo **intensivo 12–20 h/sem** (el que tu doc recomienda para lograrlo en 4–6 meses) . Si haces menos horas, mantén el mismo orden, pero baja el volumen (yo te dejo la versión “15 h/sem” por defecto).

---

1) Stack exacto (lo que vas a usar siempre)

Tu “stack 2026 empleable” queda así (no invento: está tal cual en tu documento): **Python + SQL + Git/Linux/entornos + estadística aplicada + XGBoost/LightGBM + SHAP + FastAPI + Docker + MLflow + tests + CI + monitoreo + (dbt + 1 warehouse) + (RAG con LlamaIndex + LangChain)** .
Y tu ruta en fases será: **fundamentos → ML industria → MLOps/producción (con salida mínima) → especialización (GenAI/RAG)** .

Salida mínima obligatoria del Mes 3: ✅ API FastAPI + Docker + MLflow + tests + CI .

---

2) Rutina semanal fija (para todo el plan)

> Esto evita “estudio sin entrega”.

L–V (2h/día = 10h):

- **45 min SQL** (ejercicios + guardar soluciones). “Window functions sí o sí.”
- **75 min Bloque principal** (según la semana: stats / ML / MLOps / RAG)
- **10 min cierre**: 1 commit o 1 nota (aunque sea pequeña)

Sábado (4–5h): construir proyecto (features reales, no teoría)
Domingo (30–45 min): retro + plan + pulir README/case study

---

3) Tus 3 proyectos (entregables finales)

Tu portafolio debe tener exactamente esto (lo dice tu doc):

1. **End-to-end ML en producción**: dataset → training pipeline → MLflow → FastAPI → Docker → CI → monitoreo básico
2. **Proyecto de negocio**: KPI + baseline + segmentación + análisis de error + recomendación accionable
3. **RAG/LLM app útil**: chat con docs + evaluación (precision/recall retrieval) + control de fuentes
   Y **1–2 páginas por proyecto** (case study) .

Regla: **“Un portafolio > 10 certificados”** .

---

4) Calendario día-a-día (24 semanas)

MES 1 — Fundamentos pro + SQL serio (Semanas 1–4)

Meta del mes: “que tus notebooks se vuelvan trabajo profesional” y dejar SQL fuerte.

Semana 1 — Setup profesional + SQL base fuerte

Deliverable: repo listo + 15 queries documentadas.

- **Lun:** (SQL) joins + GROUP BY. (Main) crea repo, estructura `/src /notebooks /data /docs` .
- **Mar:** (SQL) subqueries. (Main) entorno (conda/uv/poetry) + `requirements` /lock.
- **Mié:** (SQL) CTEs. (Main) plantilla README (problema → datos → métrica → cómo correr).
- **Jue:** (SQL) limpieza de datos con SQL. (Main) notebook template “EDA limpio”.
- **Vie:** (SQL) casos de negocio (ventas/cohortes). (Main) Git básico: branches + PR + commits.
- **Sáb:** mini-reto: 10 preguntas de negocio resueltas en SQL + README.
- **Dom:** revisión: ¿puedes explicar 3 queries “como a un PM”?

Semana 2 — SQL avanzado (window functions) a fuego

Deliverable: 30–40 ejercicios SQL + explicación.

- **Lun:** (SQL) `ROW_NUMBER/RANK` . (Main) crear dataset pequeño en Postgres/DuckDB.
- **Mar:** (SQL) `LAG/LEAD` . (Main) cohortes y retención.
- **Mié:** (SQL) running totals (`SUM() OVER`). (Main) performance básico (filtros temprano).
- **Jue:** (SQL) particiones por grupo (`PARTITION BY`). (Main) escribe “SQL cheatsheet v1”.

- **Vie:** (SQL) combinación: CTE + windows. (Main) 1 post corto: “3 patrones windows”.
- **Sáb:** “SQL sprint”: 20 ejercicios seguidos + subir soluciones.
- **Dom:** retro: lista de errores frecuentes + cómo evitarlos.

Semana 3 — Estadística aplicada (A/B + inferencia)

Deliverable: notebook “AB test” con decisión final.

- **Lun:** (SQL) windows repaso. (Main) probabilidad + distribuciones (ejemplos con Python).
- **Mar:** (SQL) 3 ejercicios. (Main) intervalos de confianza + interpretación.
- **Mié:** (SQL) 3 ejercicios. (Main) pruebas de hipótesis + p-value (sin dogmas).
- **Jue:** (SQL) 3 ejercicios. (Main) A/B test end-to-end (métrica, supuestos, decisión).
- **Vie:** (SQL) 3 ejercicios. (Main) leakage/sesgos: checklist para ML.
- **Sáb:** simula 2 escenarios A/B (efecto pequeño vs grande) + power básico.
- **Dom:** escribe 1 página: “Cómo decido si lanzar experimento”.

Semana 4 — Scikit-learn “bien hecho” (pipeline real)

Deliverable: baseline ML reproducible (Proyecto 1 inicia).

- **Lun:** (SQL) 3 ejercicios. (Main) Pipelines + ColumnTransformer.
- **Mar:** (SQL) 3 ejercicios. (Main) métricas: clasificación vs regresión (elige 1 problema).
- **Mié:** (SQL) 3 ejercicios. (Main) split correcto (CV / time split).
- **Jue:** (SQL) 3 ejercicios. (Main) baseline (logreg / RF) + evaluación.
- **Vie:** (SQL) 3 ejercicios. (Main) define “qué decisión de negocio habilita tu modelo”.
- **Sáb:** setup Proyecto 1: data ingestion + EDA + baseline guardado.
- **Dom:** retro + plan Mes 2.

MES 2 — ML industria (Proyecto 1 tabular potente) (Semanas 5–8)

Meta: pasar de “entreno modelos” a “construyo soluciones” .

Semana 5 — Feature engineering + baseline serio

Deliverable: baseline v1 + error analysis v1.

- **Lun:** features temporales/ratios.
- **Mar:** manejo de nulos/outliers + validación.
- **Mié:** desbalance (si aplica) + métricas por segmento.

- **Jue:** “leakage hunt”: revisa columnas sospechosas.
- **Vie:** tabla de resultados (baseline vs mejoras).
- **Sáb:** refactor: pasa de notebook a `/src` con funciones.
- **Dom:** README: problema → métrica → baseline.

Semana 6 — XGBoost/LightGBM (imprescindible en tabular)

Deliverable: modelo v2 + tracking simple.

- **Lun:** configurar XGBoost/LightGBM + early stopping
- **Mar:** tuning pequeño (5–10 configs) con criterio.
- **Mié:** calibración si aplica / thresholding orientado a negocio.
- **Jue:** métricas por segmento + fairness checks (básico).
- **Vie:** define “qué harías en producción si baja X métrica”.
- **Sáb:** refactor + config files (yaml) para reproducibilidad.
- **Dom:** “model card” 1 página (qué hace, límites, riesgos).

Semana 7 — Interpretabilidad + análisis de error profundo

Deliverable: SHAP + informe de fallos.

- **Lun:** SHAP global (top features).
- **Mar:** SHAP local (casos que fallan).
- **Mié:** análisis de error por segmento (quién sufre más).
- **Jue:** plan de nuevas features (2–3) basado en errores.
- **Vie:** re-entreno + comparación.
- **Sáb:** “storytelling”: 6 gráficas clave + conclusiones.
- **Dom:** case study v1 (1 página).

Semana 8 — “Cierre ML” (listo para producción)

Deliverable: Proyecto 1 listo para MLOps.

- **Lun:** limpieza repo + `train.py`.
- **Mar:** `predict.py` offline + validación inputs.
- **Mié:** scripts reproducibles + seeds + logging.
- **Jue:** documentación de datos + diccionario de variables.
- **Vie:** checklist final: leakage/time split ok.
- **Sáb:** release v1.0 (tag) + README final ML.
- **Dom:** retro + plan Mes 3.

MES 3 — MLOps / Producción (Proyecto 1 a producción) (Semanas 9–12)

Meta: poder decir “lo desplegué, lo monitoreé y lo reproduzco” .

Semana 9 — MLflow (tracking/registry)

Deliverable: runs en MLflow + modelo registrado.

- **Lun:** MLflow tracking local.
- **Mar:** log params/metrics/artifacts.
- **Mié:** model registry (versionado).
- **Jue:** `train.py` registra todo automáticamente.
- **Vie:** “experiment report” (tabla de runs).
- **Sáb:** refactor + documentación “cómo reproducir”.
- **Dom:** retro.

Semana 10 — FastAPI (serving)

Deliverable: API `/predict` funcionando.

- **Lun:** estructura FastAPI + pydantic schema.
- **Mar:** endpoint predict + carga de modelo.
- **Mié:** validación + mensajes de error decentes.
- **Jue:** docs automáticas + ejemplos curl.
- **Vie:** pruebas básicas del endpoint.
- **Sáb:** demo local (video opcional).
- **Dom:** retro.

Semana 11 — Docker + testing (pytest + data tests)

Deliverable: contenedor + tests automatizados.

- **Lun:** Dockerfile + build.
- **Mar:** docker-compose (API + MLflow si aplica).
- **Mié:** pytest unit tests (features, validaciones).
- **Jue:** data tests (rangos, nulos, categorías).
- **Vie:** `make test` / `make run`.
- **Sáb:** “hardening”: timeouts, logs, estructura final.
- **Dom:** retro.

Semana 12 — CI + monitoreo básico (drift / data quality)

Deliverable: CI en GitHub Actions + monitoreo simple.

- **Lun:** GitHub Actions: lint + tests + build.
- **Mar:** medir estadísticos de inputs (mean/std/cats).
- **Mié:** drift básico (comparar distribuciones).
- **Jue:** log de predicciones + latencia.
- **Vie:** documento “operación”: qué mirar si falla.
- **Sáb:** release v2.0 (Proyecto 1 final) ✅ salida mínima cumplida
- **Dom:** retro + plan Mes 4.

MES 4 — Proyecto 2 (negocio + KPI + experimento) + BI (Semanas 13–16)

Meta: demostrar que tomas decisiones de negocio con datos .

Semana 13 — Definir caso, KPI y baseline

Deliverable: 1-pager de negocio.

- **Lun:** elegir caso (churn/fraude/demanda/credit scoring)
- **Mar:** KPI primario/secundarios + definición operacional.
- **Mié:** baseline (SQL + notebook).
- **Jue:** segmentación (cohortes) + hipótesis.
- **Vie:** plan de experimento (A/B o cuasi-experimento simple).
- **Sáb:** análisis inicial + primeras conclusiones.
- **Dom:** retro.

Semana 14 — Análisis de error + recomendación accionable

Deliverable: memo de recomendación.

- **Lun:** métricas por segmento.
- **Mar:** “por qué falla” (data quality, outliers, definiciones).
- **Mié:** escenarios (sensibilidad) y trade-offs.
- **Jue:** recomendación: qué harías mañana si fueras el dueño del KPI.
- **Vie:** escribir memo (1–2 páginas).
- **Sáb:** refinar gráficos + storytelling.
- **Dom:** retro.

Semana 15 — Dashboard (Power BI/Looker/Tableau) + SQL

Deliverable: dashboard + guía de lectura.

- **Lun:** define layout del dashboard (KPI + breakdown + alertas).
- **Mar:** modelado simple para BI (tabla hechos/dimensiones).
- **Mié:** medidas (DAX/SQL) + validación.
- **Jue:** dashboard v1.
- **Vie:** “guía”: cómo interpretar y qué decisión se toma.
- **Sáb:** dashboard final + screenshots en README.
- **Dom:** retro.

Semana 16 — Case study final Proyecto 2

Deliverable: case study final (1–2 páginas)

- **Lun–Vie:** pulir escritura + evidencia + limitaciones + siguientes pasos
- **Sáb:** release Proyecto 2
- **Dom:** retro + plan Mes 5

MES 5 — Proyecto 3: GenAI / RAG (Semanas 17–20)

Meta: diferenciarte en 2026 con RAG + evaluación , usando LlamaIndex/LangChain .

Semana 17 — RAG base (ingesta → embeddings → retrieval → respuesta)

Deliverable: demo que responde con fuentes.

- **Lun:** elige corpus (PDFs/docs) y define casos de uso.
- **Mar:** chunking + embeddings + vector store.
- **Mié:** retrieval + respuestas con citas.
- **Jue:** mejorar prompt + formato de respuesta (con fuentes).
- **Vie:** logging de consultas.
- **Sáb:** demo funcional (CLI o app simple).
- **Dom:** retro.

Semana 18 — Evaluación (lo que te hace “pro”)

Deliverable: eval harness + métricas.

- **Lun:** construir set de 30–50 preguntas + “pasaje correcto”.
- **Mar:** medir recall@k (retrieval).
- **Mié:** medir “fidelidad de citas” (si usa fuentes correctas).
- **Jue:** error analysis (por qué falla: chunking, embeddings, query).
- **Vie:** ajustes + re-evaluación.
- **Sáb:** reporte de evaluación en README.
- **Dom:** retro.

Semana 19 — App usable + observabilidad

Deliverable: UI con panel de fuentes.

- **Lun:** Streamlit/Gradio: chat + sidebar de fuentes.
- **Mar:** caching + límites (tamaño docs).
- **Mié:** logs: latencia + tasa “no sé”.
- **Jue:** guardrails básicos (si no hay evidencia, no inventar).
- **Vie:** pruebas “smoke”.
- **Sáb:** demo final.
- **Dom:** retro.

Semana 20 — Case study Proyecto 3 + release

Deliverable: case study (1–2 páginas) + release.

- **Lun–Vie:** pulir: problema → enfoque → métricas (incluye recall@k) → decisión
- **Sáb:** release Proyecto 3
- **Dom:** retro + plan Mes 6

MES 6 — Empleo: entrevistas + aplicaciones + marca (Semanas 21–24)

Meta: convertir tu trabajo en **ofertas** (con pipeline).

Semana 21 — Portafolio final (landing)

Deliverable: README principal + enlaces + demos.

- **Lun:** README “home” con 3 proyectos y 2 líneas de impacto c/u.
- **Mar:** ordenar repos + capturas + diagramas simples.
- **Mié:** pulir instrucciones “1 comando para correr”.

- **Jue:** checklist de calidad (tests/CI pasan).
- **Vie:** escribir post corto por proyecto (LinkedIn/GitHub).
- **Sáb:** grabar demo 2–4 min por proyecto (opcional).
- **Dom:** retro.

Semana 22 — CV + LinkedIn “orientado a evidencia”

Deliverable: CV final + LinkedIn optimizado.

- **Lun:** CV: bullets con impacto (métrica + qué hiciste + stack).
- **Mar:** sección “Proyectos” con links y resultados.
- **Mié:** LinkedIn: about + featured + keywords (SQL/FastAPI/Docker/MLflow/RAG).
- **Jue:** adaptar CV a 2 perfiles: DS (negocio) vs ML Eng (producción)
- **Vie:** 10 mensajes de networking (plantilla corta).
- **Sáb:** aplicar a 15–25 vacantes.
- **Dom:** retro.

Semana 23 — Rutina entrevistas (SQL + ML + MLOps + RAG)

Deliverable: 5 mocks + banco de respuestas.

- **Lun:** SQL: windows/CTEs (1 hora) + 1 mock.
- **Mar:** ML: leakage/validación/métricas + 1 mock.
- **Mié:** MLOps: explicar tu API+CI+monitoreo + 1 mock.
- **Jue:** RAG: evaluación recall@k + control fuentes + 1 mock.
- **Vie:** “story”: 3 proyectos en 10 minutos.
- **Sáb:** aplicar + retro de entrevistas.
- **Dom:** descanso ligero.

Semana 24 — Aplicación masiva inteligente + cierre

Deliverable: 40–60 aplicaciones totales + seguimiento.

- **Lun–Vie:** 5–10 aplicaciones/día + 3 follow-ups/día (hoja de tracking).
- **Sáb:** ajustar CV/portafolio según feedback real.
- **Dom:** plan del mes siguiente (si aún no hay oferta: iterar).

---

5) Tablero tipo Trello (para que lo ejecutes sin pensar)

Crea 5 columnas:

1. **Backlog (mes)**
2. **Esta semana**
3. **Hoy**
4. **Bloqueado**
5. **Hecho**

Regla: *cada día debe mover algo a "Hecho"* (aunque sea pequeño: una query, un test, un endpoint).

---

6) Checklist imprimible (lo mínimo que debes poder decir en entrevistas)

Cuando te pregunten "¿qué sabes hacer?", tú debes poder demostrar:

SQL (obligatorio)

- window functions, CTEs, subqueries

ML industria

- XGBoost/LightGBM + validación correcta + SHAP + error analysis

Producción / MLOps (diferencial)

- FastAPI + Docker + MLflow + tests + CI + monitoreo drift/data quality

GenAI (diferenciador 2026)

- RAG con LlamaIndex + LangChain + evaluación (precision/recall retrieval)

Si quieres, en el siguiente mensaje te dejo **plantillas listas para copiar/pegar**:

1. estructura de repo (carpetas + naming),
2. plantilla de README "que contrata",
3. plantilla de case study 1–2 páginas,
4. y una hoja de tracking de aplicaciones (campos exactos).
   Aquí tienes tu **plan universitario (6 meses / 24 semanas)** con **tareas día por día**, entregables y "evaluación", basado en tu guía: **SQL serio + estadística aplicada + ML de industria + MLOps/producción + GenAI/RAG + portafolio end-to-end**.

0) “Sílabo” (formato universidad)

Curso

Bootcamp Personal: Data Scientist / Applied ML (Job-Ready en 6 meses)

Carga horaria (realista)

- Objetivo: **12–20 h/sem** (intensivo).
- Si haces menos, mantén el orden y baja volumen (sin cambiar fases).

Stack obligatorio (lo que vas a usar “de verdad”)

Python + SQL + Git/Linux + estadística aplicada + XGBoost/LightGBM + SHAP + FastAPI + Docker + MLflow + tests + CI + monitoreo + (dbt + 1 warehouse) + (RAG con LlamaIndex/LangChain).

Entregables finales (3 proyectos + case studies)

1. **Proyecto 1:** end-to-end ML en producción (training→MLflow→API→Docker→CI→monitoreo básico)
2. **Proyecto 2:** negocio (KPI→baseline→segmentación→decisión/recomendación)
3. **Proyecto 3:** RAG útil (chat con docs + **evaluación** precision/recall retrieval + control de fuentes)

- **1–2 páginas por proyecto** (case study).

Regla de oro (para que sí te contraten)

- **Cada semana** debe salir **algo visible** (repo/queries/notebook/endpoint/MLflow run/README).
- **Cada día** debe mover algo a “Hecho” (aunque sea pequeño).

Evaluación (tu “nota”)

- 40%: entregables semanales (evidencia en GitHub)
- 50%: Proyectos 1–3 (con case studies + reproducibilidad)
- 10%: preparación laboral (CV/LinkedIn/mocks/aplicaciones)

---

1) Rutina diaria fija (esto NO se negocia)

L–V (2h/día):

- **45 min SQL** (ejercicios + guardar soluciones)
- **75 min Bloque principal** (stats / ML / MLOps / RAG según semana)
- **10 min cierre:** 1 commit o 1 nota (aunque sea pequeña).

Sábado (4–5h): construir proyecto (features reales, no teoría).
Domingo (30–45m): retro + plan + pulir README/case study.

---

2) Calendario día-a-día (24 semanas) empezando hoy

Inicio asumido: Lun **2026-03-02** → Termina Dom **2026-08-16** (24 semanas).

> En cada día te pongo: **SQL (45m)** + **Main (75m)** + **Evidencia (10m)**.

MES 1 (Semanas 1–4) — Fundamentos pro + SQL serio

Meta: notebooks “nivel trabajo” + SQL fuerte.

Semana 1 (2026-03-02 → 03-08) — Setup profesional + SQL base

Deliverable: repo listo + 15 queries documentadas.

Día 1 (Lun): SQL joins+GROUP BY | Main: crear repo y estructura /src /notebooks /data /docs | Evidencia: primer README + commit.
Día 2 (Mar): SQL subqueries | Main: entorno (conda/uv/poetry) + requirements/lock | Evidencia: “cómo correr” en README.
Día 3 (Mié): SQL CTEs | Main: plantilla README (problema→datos→métrica→cómo correr) | Evidencia: template.md.
Día 4 (Jue): SQL limpieza con SQL | Main: notebook template “EDA limpio” | Evidencia: notebook_base.ipynb.
Día 5 (Vie): SQL casos negocio (ventas/cohortes) | Main: Git (branches+PR+commits) | Evidencia: PR con cambios.
Día 6 (Sáb): Construcción (4–5h): mini-reto 10 preguntas de negocio en SQL + README | Evidencia: /notes/sql + capturas.
Día 7 (Dom): Retro (30–45m): explica 3 queries “como a PM” + plan semana 2 | Evidencia: notas de retro.

Semana 2 (2026-03-09 → 03-15) — SQL avanzado (window functions)

Deliverable: 30–40 ejercicios SQL + explicación.

Día 8 (Lun): SQL ROW_NUMBER/RANK | Main: dataset pequeño en Postgres/DuckDB | Evidencia: script carga datos.
Día 9 (Mar): SQL LAG/LEAD | Main: cohortes y retención | Evidencia: 3 queries comentadas.
Día 10 (Mié): SQL running totals | Main: performance básico (filtros temprano) | Evidencia: “antes/después” query.
Día 11 (Jue): SQL PARTITION BY | Main: escribir “SQL cheatsheet v1” | Evidencia: cheatsheet.md.
Día 12 (Vie): SQL mezcla windows+CTE | Main: 10 ejercicios tipo entrevista | Evidencia: 10 queries + explicación (3 bullets c/u).
Día 13 (Sáb): SQL sprint (4–5h): completar 30–40 ejercicios + ordenar por temas | Evidencia: índice + tags en notas.
Día 14 (Dom): Retro: lista de errores frecuentes + plan semana 3 | Evidencia: checklist personal.

Semana 3 (2026-03-16 → 03-22) — Estadística aplicada (A/B + inferencia)

Deliverable: notebook “AB test” con decisión final.
(temas: p-values, intervalos, power, etc.)

Día 15: SQL 3 ejercicios windows | Main: distribuciones + simulaciones | Evidencia: notebook stats_01.
Día 16: SQL 3 ejercicios | Main: intervalos de confianza + interpretación | Evidencia: notas “cómo lo explico a negocio”.
Día 17: SQL 3 ejercicios | Main: hipótesis + p-value (sin dogmas) | Evidencia: ejemplos con conclusiones.
Día 18: SQL 3 ejercicios | Main: diseño A/B (métrica, supuestos) | Evidencia: plantilla A/B.
Día 19: SQL 3 ejercicios | Main: power básico + tamaño muestra | Evidencia: función power en Python.
Día 20 (Sáb): Construcción: simula 2 escenarios A/B + escribe decisión recomendada | Evidencia: AB_testing_from_scratch.ipynb.
Día 21 (Dom): Retro: 1 página “cómo decido si lanzo experimento” | Evidencia: memo.md.

Semana 4 (2026-03-23 → 03-29) — Scikit-learn “bien hecho” (pipeline real)

Deliverable: baseline ML reproducible (inicia Proyecto 1).

Día 22: SQL 3 ejercicios | Main: Pipelines + ColumnTransformer | Evidencia: pipeline.py.
Día 23: SQL 3 ejercicios | Main: métricas ligadas a negocio (elige problema) | Evidencia: métricas.md.
Día 24: SQL 3 ejercicios | Main: split correcto (CV vs time split) | Evidencia: validación.py.
Día 25: SQL 3 ejercicios | Main: baseline (logreg/RF) + evaluación | Evidencia: baseline.ipynb.
Día 26: SQL 3 ejercicios | Main: define decisión de negocio que habilita el modelo | Evidencia: decision.md.
Día 27 (Sáb): Construcción: setup Proyecto 1 (ingesta+EDA+baseline guardado) | Evidencia: train_baseline + artifact.
Día 28 (Dom): Retro + plan MES 2 | Evidencia: backlog semana 5.

MES 2 (Semanas 5–8) — ML “de industria” (Proyecto 1 tabular potente)

Meta: pasar de “entreno modelos” a “construyo soluciones”.

Semana 5 (2026-03-30 → 04-05) — Feature engineering + baseline serio

Deliverable: baseline v1 + error analysis v1.

Día 29: SQL 3 ejercicios | Main: features temporales/ratios | Evidencia: features_v1.py.
Día 30: SQL 3 ejercicios | Main: nulos/outliers + validación | Evidencia: data_checks.md.
Día 31: SQL 3 ejercicios | Main: desbalance + métricas por segmento | Evidencia: report_segmentos.ipynb.
Día 32: SQL 3 ejercicios | Main: “leakage hunt” (columnas sospechosas) | Evidencia: checklist_leakage.md.
Día 33: SQL 3 ejercicios | Main: tabla resultados baseline vs mejoras | Evidencia: experiments.csv.
Día 34 (Sáb): Construcción: refactor notebook→/src (funciones) | Evidencia: estructura limpia + README baseline.
Día 35 (Dom): Retro: README “problema→métrica→baseline” | Evidencia: README actualizado.

Semana 6 (2026-04-06 → 04-12) — XGBoost/LightGBM

Deliverable: modelo v2 + tracking simple.

Día 36: SQL 3 ejercicios | Main: configurar XGB/LGBM + early stopping | Evidencia: training_config.yaml.

Día 37: SQL 3 ejercicios | Main: tuning 5–10 configs con criterio | Evidencia: tabla tuning.
Día 38: SQL 3 ejercicios | Main: calibración/thresholding orientado a negocio | Evidencia: thresholds.md.
Día 39: SQL 3 ejercicios | Main: métricas por segmento + fairness básico | Evidencia: fairness_notes.md.
Día 40: SQL 3 ejercicios | Main: plan de operación “si baja X métrica” | Evidencia: ops_plan.md.
Día 41 (Sáb): Construcción: refactor + configs reproducibles (yaml) | Evidencia: run reproducible.
Día 42 (Dom): Retro: “model card” 1 página | Evidencia: model_card.md.

Semana 7 (2026-04-13 → 04-19) — Interpretabilidad + error analysis

Deliverable: SHAP + informe de fallos.

Día 43: SQL 3 ejercicios | Main: SHAP global (top features) | Evidencia: shap_global.png.
Día 44: SQL 3 ejercicios | Main: SHAP local (casos que fallan) | Evidencia: shap_local_cases.md.
Día 45: SQL 3 ejercicios | Main: error analysis por segmento | Evidencia: segmentos_fail.ipynb.
Día 46: SQL 3 ejercicios | Main: plan 2–3 features nuevas (basado en errores) | Evidencia: backlog_features.md.
Día 47: SQL 3 ejercicios | Main: reentreno + comparación | Evidencia: report_compare.md.
Día 48 (Sáb): Construcción: storytelling (6 gráficas + conclusiones) | Evidencia: slides.md / figuras.
Día 49 (Dom): Retro: case study v1 (1 página) | Evidencia: case_study_v1.md.

Semana 8 (2026-04-20 → 04-26) — Cierre ML (listo para producción)

Deliverable: Proyecto 1 listo para MLOps + release v1.0.

Día 50: SQL 3 ejercicios | Main: limpieza repo + train.py | Evidencia: train.py ejecuta end-to-end.
Día 51: SQL 3 ejercicios | Main: predict.py offline + validación inputs | Evidencia: validación schema.
Día 52: SQL 3 ejercicios | Main: seeds + logging reproducible | Evidencia: logs + misma métrica.
Día 53: SQL 3 ejercicios | Main: diccionario de variables + doc datos | Evidencia: data_dictionary.md.
Día 54: SQL 3 ejercicios | Main: checklist leakage/time split OK | Evidencia: checklist.md.
Día 55 (Sáb): Construcción: tag release v1.0 + README final ML | Evidencia: release notes.
Día 56 (Dom): Retro + plan MES 3 | Evidencia: backlog semana 9.

MES 3 (Semanas 9–12) — MLOps / Producción (Proyecto 1 a producción)

Salida mínima obligatoria: API FastAPI + Docker + MLflow + tests + CI.

Semana 9 (2026-04-27 → 05-03) — MLflow tracking/registry

Deliverable: runs en MLflow + modelo registrado.

Día 57: SQL 3 ejercicios | Main: MLflow local tracking | Evidencia: mlruns/ + screenshot.
Día 58: SQL 3 ejercicios | Main: log params/metrics/artifacts | Evidencia: run completo.
Día 59: SQL 3 ejercicios | Main: model registry (versionado) | Evidencia: model v1 registrado.
Día 60: SQL 3 ejercicios | Main: train.py registra automático | Evidencia: 1 comando reproduce.
Día 61: SQL 3 ejercicios | Main: “experiment report” (tabla runs) | Evidencia: report.md.
Día 62 (Sáb): Refactor + doc “cómo reproducir” | Evidencia: README reproducible.
Día 63 (Dom): Retro + plan FastAPI | Evidencia: issues para semana 10.

Semana 10 (2026-05-04 → 05-10) — FastAPI serving

Deliverable: API /predict funcionando.

Día 64: SQL 3 ejercicios | Main: estructura FastAPI + pydantic schema | Evidencia: app arranca.
Día 65: SQL 3 ejercicios | Main: endpoint /predict + carga modelo | Evidencia: curl responde.
Día 66: SQL 3 ejercicios | Main: validación + errores decentes | Evidencia: ejemplos 400/422.
Día 67: SQL 3 ejercicios | Main: docs automáticas + ejemplos curl | Evidencia: docs en /docs.
Día 68: SQL 3 ejercicios | Main: pruebas básicas endpoint | Evidencia: test_api.py.
Día 69 (Sáb): Demo local (video opcional) | Evidencia: demo.md.
Día 70 (Dom): Retro + plan Docker/tests | Evidencia: checklist semana 11.

Semana 11 (2026-05-11 → 05-17) — Docker + testing

Deliverable: contenedor + tests automatizados.

Día 71: SQL 3 ejercicios | Main: Dockerfile + build | Evidencia: docker build OK.
Día 72: SQL 3 ejercicios | Main: docker-compose (API + MLflow si aplica) | Evidencia: stack levanta.
Día 73: SQL 3 ejercicios | Main: pytest unit tests (features/validaciones) | Evidencia: tests pasan.
Día 74: SQL 3 ejercicios | Main: data tests (rangos, nulos, categorías) | Evidencia: data_tests.py.
Día 75: SQL 3 ejercicios | Main: make test / make run | Evidencia: 1 comando corre todo.

Día 76 (Sáb): Hardening: timeouts, logs, estructura final | Evidencia: logging + healthcheck.
Día 77 (Dom): Retro + plan CI/monitoreo | Evidencia: tareas semana 12.

Semana 12 (2026-05-18 → 05-24) — CI + monitoreo básico (drift/data quality)

Deliverable: CI en GitHub Actions + monitoreo simple.

Día 78: SQL 3 ejercicios | Main: GitHub Actions (lint+tests+build) | Evidencia: badge CI.
Día 79: SQL 3 ejercicios | Main: stats de inputs (mean/std/cats) | Evidencia: metrics_inputs.json.
Día 80: SQL 3 ejercicios | Main: drift básico (comparar distribuciones) | Evidencia: drift_report.md.
Día 81: SQL 3 ejercicios | Main: log predicciones + latencia | Evidencia: logs + tiempos.
Día 82: SQL 3 ejercicios | Main: doc “operación” (qué mirar si falla) | Evidencia: 운영.md/ops.md.
Día 83 (Sáb): Release v2.0 (Proyecto 1 completo) | Evidencia: release notes.
Día 84 (Dom): Retro + plan Proyecto 2 | Evidencia: backlog semana 13.

MES 4 (Semanas 13–16) — Proyecto 2 (negocio + KPI + experimento) + BI

Meta: demostrar decisiones de negocio con datos.

Semana 13 (2026-05-25 → 05-31) — Definir caso, KPI y baseline

Deliverable: 1-pager de negocio.

Día 85: SQL: queries del caso | Main: elegir caso (churn/fraude/demanda/credit scoring) | Evidencia: problem.md.
Día 86: SQL: KPI queries | Main: KPI primario/secundarios (definición operacional) | Evidencia: kpi_definition.md.
Día 87: SQL: baseline KPI | Main: baseline (SQL + notebook) | Evidencia: baseline_kpi.ipynb.
Día 88: SQL: cohortes | Main: segmentación + hipótesis | Evidencia: hipótesis.md.
Día 89: SQL: validación | Main: plan de experimento (A/B o cuasi-experimento) | Evidencia: experiment_plan.md.
Día 90 (Sáb): Análisis inicial + primeras conclusiones | Evidencia: 1-pager v1.
Día 91 (Dom): Retro | Evidencia: backlog semana 14.

Semana 14 (2026-06-01 → 06-07) — Recomendación accionable (memo)

Deliverable: memo 1–2 páginas.

Día 92: SQL: segmentación | Main: métricas por segmento | Evidencia: tabla segmentos.
Día 93: SQL: outliers | Main: “por qué falla” (data quality/definiciones) | Evidencia: riesgos.md.
Día 94: SQL: sensibilidad | Main: escenarios + trade-offs | Evidencia: sensitivity.ipynb.
Día 95: SQL: supporting queries | Main: recomendación (“qué haría mañana si fuera dueño del KPI”) | Evidencia: reco.md.
Día 96: SQL: consolidación | Main: escribir memo final | Evidencia: memo.pdf/md.
Día 97 (Sáb): Refinar gráficos + storytelling | Evidencia: figuras + narrativa.
Día 98 (Dom): Retro | Evidencia: backlog semana 15.

Semana 15 (2026-06-08 → 06-14) — Dashboard (BI)

Deliverable: dashboard + guía de lectura.

Día 99: SQL: modelo de datos | Main: layout (KPI+breakdowns+alertas) | Evidencia: wireframe.
Día 100: SQL: dims/facts | Main: modelado simple (hechos/dimensiones) | Evidencia: schema.png.
Día 101: SQL: medidas | Main: medidas DAX/SQL + validación | Evidencia: medidas.md.
Día 102: SQL: checks | Main: dashboard v1 | Evidencia: dashboard_v1.
Día 103: SQL: edge cases | Main: guía “cómo interpretar/decidir” | Evidencia: guia_dashboard.md.
Día 104 (Sáb): Dashboard final + screenshots en README | Evidencia: README con capturas.
Día 105 (Dom): Retro | Evidencia: backlog semana 16.

Semana 16 (2026-06-15 → 06-21) — Case study Proyecto 2 + release

Deliverable: case study final + release.

Días 106–110 (Lun–Vie): SQL 30–45m diarios | Main: pulir escritura (problema→método→métrica→decisión→limitaciones) | Evidencia: case_study_vFinal.md.
Día 111 (Sáb): Release Proyecto 2 | Evidencia: tag + README final.
Día 112 (Dom): Retro + plan Mes 5 | Evidencia: backlog semana 17.

MES 5 (Semanas 17–20) — Proyecto 3 GenAI (RAG) + evaluación

Meta: diferenciarte con RAG + evaluación (no solo demo).

Semana 17 (2026-06-22 → 06-28) — RAG base (pipeline completo)

Deliverable: demo que responde con fuentes.

Día 113: SQL 30–45m | Main: elegir corpus (PDFs/docs) + casos de uso | Evidencia: corpus.md.
Día 114: SQL 30–45m | Main: chunking + embeddings + vector store | Evidencia: ingest.py.
Día 115: SQL 30–45m | Main: retrieval + respuestas con citas | Evidencia: demo.ipynb con fuentes.
Día 116: SQL 30–45m | Main: mejorar prompt + formato con fuentes | Evidencia: prompt.md.
Día 117: SQL 30–45m | Main: logging de consultas | Evidencia: logs + latencia.
Día 118 (Sáb): Demo funcional (CLI o app simple) | Evidencia: “1 comando” en README.
Día 119 (Dom): Retro | Evidencia: backlog semana 18.

Semana 18 (2026-06-29 → 07-05) — Evaluación de RAG (lo que te hace pro)

Deliverable: eval harness + métricas.
(armar 30–50 preguntas, medir recall@k, exactitud de citas, registrar resultados)

Día 120: SQL 30–45m | Main: set de 30–50 preguntas + “gold passages” | Evidencia: eval_set.jsonl.
Día 121: SQL 30–45m | Main: medir recall@k del retrieval | Evidencia: tabla recall@k.
Día 122: SQL 30–45m | Main: medir fidelidad/uso correcto de fuentes | Evidencia: citation_accuracy.md.
Día 123: SQL 30–45m | Main: error analysis (chunking/embeddings/query) | Evidencia: errores.md.
Día 124: SQL 30–45m | Main: re-run evaluación tras cambios | Evidencia: before_after.csv.
Día 125 (Sáb): “eval report” en README (gráficas simples + conclusiones) | Evidencia: README actualizado.
Día 126 (Dom): Retro | Evidencia: backlog semana 19.

Semana 19 (2026-07-06 → 07-12) — App usable + observabilidad

Deliverable: UI con panel de fuentes + logging (latencia, tasa “no sé”).

Día 127: SQL 30–45m | Main: Streamlit/Gradio chat + sidebar fuentes | Evidencia: app.py.
Día 128: SQL 30–45m | Main: caching + límites (tamaño docs) | Evidencia: settings.md.
Día 129: SQL 30–45m | Main: logs (latencia + tasa “no sé”) | Evidencia: metrics.log.
Día 130: SQL 30–45m | Main: guardrails (si no hay evidencia, no inventar) | Evidencia: reglas.md.
Día 131: SQL 30–45m | Main: pruebas smoke | Evidencia: test_smoke.py.
Día 132 (Sáb): Demo final (1 comando) | Evidencia: README “run locally”.
Día 133 (Dom): Retro | Evidencia: backlog semana 20.

Semana 20 (2026-07-13 → 07-19) — Case study Proyecto 3 + release

Deliverable: case study + release.

Días 134–138 (Lun–Vie): SQL 30–45m | Main: pulir case study (incluye recall@k + límites + decisiones) | Evidencia: case_study_RAG.md.
Día 139 (Sáb): Release Proyecto 3 | Evidencia: tag + changelog.
Día 140 (Dom): Retro + plan Mes 6 | Evidencia: pipeline aplicaciones.

MES 6 (Semanas 21–24) — Empleo: portafolio + CV + entrevistas + aplicaciones

Meta: convertir tu trabajo en ofertas.

Semana 21 (2026-07-20 → 07-26) — Portafolio final (landing)

Deliverable: README principal + links + demos.

Día 141: SQL 45–60m | Main: README “home” con 3 proyectos + impacto | Evidencia: home README.
Día 142: SQL 45–60m | Main: ordenar repos + capturas/diagramas | Evidencia: assets/.
Día 143: SQL 45–60m | Main: pulir “1 comando para correr” en cada proyecto | Evidencia: makefile/scripts.
Día 144: SQL 45–60m | Main: checklist calidad (tests/CI pasan) | Evidencia: badge verde.
Día 145: SQL 45–60m | Main: post corto por proyecto (GitHub/LinkedIn) | Evidencia: draft posts.
Día 146 (Sáb): Grabar demo 2–4 min (opcional) | Evidencia: links (unlisted).
Día 147 (Dom): Retro | Evidencia: backlog CV.

Semana 22 (2026-07-27 → 08-02) — CV + LinkedIn orientado a evidencia

Deliverable: CV final + LinkedIn optimizado.

Día 148: SQL 45–60m | Main: bullets con impacto (métrica + qué hiciste + stack) | Evidencia: CV v1.
Día 149: SQL 45–60m | Main: sección Proyectos con links + resultados | Evidencia: CV v2.
Día 150: SQL 45–60m | Main: LinkedIn “about” + keywords (SQL/FastAPI/Docker/MLflow/RAG) | Evidencia: perfil actualizado.
Día 151: SQL 45–60m | Main: 2 versiones CV (DS negocio vs ML Eng producción) | Evidencia: cv_ds.pdf + cv_mle.pdf.
Día 152: SQL 45–60m | Main: 10 mensajes networking (plantilla) | Evidencia: lista contactos.
Día 153 (Sáb): Aplicar a 15–25 vacantes | Evidencia: hoja tracking.
Día 154 (Dom): Retro | Evidencia: mejoras CV.

Semana 23 (2026-08-03 → 08-09) — Entrevistas técnicas (rutina diaria)

Deliverable: 5 mocks + banco de respuestas.

Día 155: SQL 1h (windows/CTEs) + 1 mock SQL | Main: repasar tus queries “como PM” | Evidencia: respuestas.md.
Día 156: SQL 1h | Main: ML (leakage/validación/métricas negocio) + 1 mock | Evidencia: banco_respuestas_ml.md.
Día 157: SQL 1h | Main: MLOps (API+CI+monitoreo) + 1 mock | Evidencia: “explicación 5 min”.
Día 158: SQL 1h | Main: RAG (eval recall@k + control fuentes) + 1 mock | Evidencia: respuestas_rag.md.
Día 159: SQL 1h | Main: “story”: 3 proyectos en 10 min | Evidencia: guion.md.
Día 160 (Sáb): Aplicar + retro de mocks | Evidencia: lista fallos + fixes.
Día 161 (Dom): Descanso ligero + retro | Evidencia: plan semana 24.

Semana 24 (2026-08-10 → 08-16) — Aplicación masiva inteligente

Deliverable: 40–60 aplicaciones + seguimiento.

Días 162–166 (Lun–Vie):

- SQL 45–60m diarios
- Main: 5–10 aplicaciones/día + 3 follow-ups/día (con tracking)
- Evidencia: hoja tracking + CV ajustado por rol.

Día 167 (Sáb): Ajustar CV/portafolio según feedback real | Evidencia: cambios y versión nueva.
Día 168 (Dom): Retro final + plan siguiente iteración (si falta oferta) | Evidencia: plan_30_dias.md.

---

3) Recursos (mínimos, alta utilidad)

Para no perderte, apóyate en estos (son los que tu doc recomienda por ROI):

- **Kaggle Learn** (Python/SQL/Pandas/ML)
- **ML Specialization (Andrew Ng)** para estructura mental
- **MLOps Zoomcamp** + **Machine Learning in Production** (para producción real)
- **RAG con LlamaIndex + LangChain** + **HuggingFace course**

---

4) Cómo llevar esto “sin que te robe tiempo” (backlog simple)

Tu documento propone un tablero simple y funcional: **5 columnas** (y listo).

- Backlog (mes)
- Esta semana
- Hoy
- Bloqueado
- Hecho

Regla: **NO borres Hecho**; archívalo por semana/mes para que sea evidencia (y motivación). Y recuerda: *cada día mueve algo a Hecho*.

Aquí tienes **reglas claras (tipo “contrato”)** para que el plan sí se cumpla en 6 meses, estudiando solo. Las reglas están alineadas con tu guía: **entregables visibles cada semana**, **SQL diario**, **portafolio > certificados**, y **enfocarte en producción** (FastAPI/Docker/MLflow/tests/CI/monitoreo).

Reglas de ejecución (diarias y semanales)

1) Regla de “algo a Hecho” (diaria)

Cada día debe quedar una evidencia, aunque sea pequeña:

- 1 query SQL guardada y comentada
- 1 commit
- 1 test nuevo
- 1 sección de README/case study
  Tu guía lo plantea como hábito: que cada día muevas algo a “Hecho”.

2) SQL es obligatorio 5 días/semana

45–60 min de SQL L–V (aunque estés cansado). Tu rutina fija lo pone como núcleo.
Si un día fallas, **lo recuperas el sábado** (no lo acumules).

3) Regla del “sábado de construcción”

El sábado no es teoría: es **build (4–5h)** en el proyecto de la fase.
Si no construyes sábado, tu portafolio se muere.

4) Regla del “domingo de cierre”

Domingo (30–45 min): **retro + plan** y pulir README/case study.
Sin domingo, la semana siguiente arranca sin dirección.

5) Regla de “una semana = un entregable visible”

Cada semana debe terminar con **algo publicable** (GitHub/README/demo). Tu guía insiste en “salidas visibles” semanales.
Si una semana no deja evidencia, es como si no hubieras estudiado.

Reglas de foco (para no dispersarte)

6) Máximo 2 cosas “en progreso”

No abras 5 cursos a la vez.
WIP = 2: SQL + el bloque principal de la semana.

7) Portafolio > certificados

Prohibido coleccionar certificados si no hay entregables.
Tu doc lo dice explícito: **“Un portafolio > 10 certificados.”**

8) Tu prioridad es “producción”

Antes de “Deep Learning fancy”, asegúrate de cumplir la salida mínima:
✅ FastAPI + Docker + MLflow + tests + CI + monitoreo básico
Porque eso es lo que te diferencia.

9) Un solo stack por 6 meses

Elige y no cambies:

- 1 lenguaje (Python)
- 1 librería tabular principal (sklearn + XGBoost/LightGBM)
- 1 framework API (FastAPI)
- 1 tracker (MLflow)
- 1 dashboard (Power BI/Looker/Tableau)
  Cambiar herramientas a mitad mata el progreso.

Reglas de calidad (para que el portafolio sí convenza)

10) Regla de reproducibilidad

Todo proyecto debe correr con:

- instrucciones claras (“1 comando” o pasos mínimos)
- seeds / config
- resultados repetibles
  Tu guía orienta a “salidas de producción” y evidencias reales.

11) Regla de documentación mínima

Cada entrega importante debe tener:

- README con problema → datos → métrica → resultados → cómo correr
- 3–5 gráficos clave o tabla de métricas
- limitaciones y próximos pasos
  (En especial porque tu guía pide **1–2 páginas por proyecto**).

12) Regla “sin notebooks sucios”

Notebook solo para exploración.
Lo que cuenta va en `/src` como código modular (funciones/scripts).

13) Regla de evaluación real (RAG)

Si haces RAG, no basta demo: debes medir retrieval (recall@k / calidad de citas). Tu guía lo exige.

Reglas de control (para que no te engañes)

14) KPI semanal personal

Cada domingo responde:

- ¿Qué evidencia quedó esta semana?
- ¿Qué aprendí (3 bullets)?
- ¿Qué me bloqueó y cómo lo arreglo?

15) Si te atrasas: recorta volumen, no fases

No cambies el orden del plan.
Recorta “extras”, pero mantén la ruta: SQL → stats → ML → MLOps → negocio → RAG → empleo.

16) Regla de “dos días malos seguidos = reset”

Si fallas 2 días seguidos:

- Día 3 haces **solo lo mínimo**:
  - 30 min SQL
  - 30 min avanzar algo del proyecto
  - 1 commit
    La clave es **no romper la cadena**.

Reglas de búsqueda de empleo (mes 6, sin excusas)

17) Preparación laboral empieza antes

Desde semana 8: al menos 1 vez/semana:

- pulir README
- escribir 5 bullets de CV
- practicar contar tu proyecto en 2 minutos

18) Aplicación con tracking

En el mes 6:

- 5–10 aplicaciones/día + seguimiento
  Tu guía lo pone como cierre del plan.

Tu “contrato” mínimo (si solo cumples 5 reglas)

Si quieres lo mínimo que garantiza avance:

1. SQL 5 días/semana
2. Sábado build
3. Domingo cierre
4. 1 evidencia diaria
5. 1 entregable visible semanal